

2026년도 전국기능경기대회

분 과	IT · 디자인	직 종 명	클라우드컴퓨팅
경기시간	4시간 (2과제)		

○ 시행 시 유의사항

(시행 전)

1. 경기 시작 전 선수별 AWS 계정 접속 가능 여부를 확인한다.
2. 선수에게 각 모듈별 필요한 배포 파일을 제공한다.
3. 선수에게 비번호를 부여하고, 모든 리소스명은 문제지의 명명 규칙을 따르도록 안내한다. (특히 S3와 같이 Global Unique한 리소스 명을 요구할 경우 비번호를 사용한다.)
4. 경기 시작 전 CloudShell, AWS CLI와 같은 도구 사용 가능 여부를 확인한다.
5. 경기 시작 전 인터넷 접속, DNS 질의, AWS Console 로그인, CloudShell 실행 상태를 확인한다.

(시행 중)

1. 선수는 지급된 AWS 계정과 제공 파일만 사용하여 과제를 수행한다.
2. 별도의 지시가 없는 Region 선택형 AWS 리소스는 ap-northeast-2(서울) 리전에 생성한다. 단, CloudFront, IAM 등 Global 서비스는 예외로 한다.
3. 모든 리소스 이름, 태그, 환경 변수는 대소문자를 구분한다.

(시행 후)

1. 경기 종료 후 선수는 배포된 리소스 및 리소스를 임의로 수정하지 않는다.
2. 경기 종료 후 채점 위원은 선수 제출 정보를 바탕으로 CloudShell에서 채점 스크립트를 실행한다.
3. 채점 중 대기가 필요할 경우 항목당 최대 3분을 넘기지 않는다.
4. 이의신청 절차 종료 후 선수 AWS 계정에 생성된 리소스를 정리한다.

2026년도 전국기능경기대회

직 종 명	클라우드컴퓨팅	과 제 명	Small Challenges	과제번호	제2과제
경기시간	4시간	비 번 호		심사위원 확 인	(인)

1. 요구사항

제2과제 Small Challenges는 서로 독립된 4개 모듈로 구성됩니다. 각 모듈 요구사항에 명시된 리전을 준수하고, 고정 리소스 이름은 정확히 사용합니다.

1. DocumentDB based NoSQL Application
2. Simplify Service Networking with VPC Lattice
3. Cloud Event Handling
4. Event-driven Pod Scaling with AWS SQS

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하십시오.

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요 시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방하여 주시기 바랍니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 본 과제의 채점은 문제지에 명시된 고정 리소스 이름과 일부 고정 Name Tag를 기준으로 진행됩니다.
- 6) 문제지에서 명시하지 않은 리소스의 Name Tag는 자유롭게 지정할 수 있습니다.
- 7) 채점은 CloudShell에서 자동화 채점 스크립트를 실행하여 진행됩니다.
- 8) 경기 종료 후 채점 및 이의신청 절차가 완료될 때까지 생성한 리소스를 삭제하거나 임의로 수정하지 마십시오.
- 9) 리전이 잘못된 경우 해당 모듈은 0점 처리될 수 있습니다.
- 10) 3모듈 기능 검증 시 skills-ceh-protected-sg에 테스트용 Inbound 규칙 TCP/22 from 0.0.0.0/0을 임시 추가할 수 있으며, 180초 이내 제거되어야 합니다.
- 11) 4모듈은 CloudShell에서 aws eks update-kubeconfig 및 kubectl 명령으로 확인하므로 EKS Cluster Endpoint는 CloudShell에서 접근 가능해야 합니다.
- 12) 4모듈 Worker Pod는 Karpenter가 생성한 skills-sqs-nodepool 기반 EC2 Worker Node에 스케줄링되어야 합니다.

- 13) Karpenter/KEDA는 Karpenter 1.x, KEDA 2.x를 사용하고, 채점은 리소스 이름과 요구 필드 존재 여부를 기준으로 합니다.

3. DocumentDB based NoSQL Application

선수는 Amazon DocumentDB MongoDB 호환 NoSQL 서비스를 사용하여 주문, 상품, 세션 데이터를 저장하고 조회할 수 있는 데이터저장소를 구성해야 합니다. 본 모듈은 ap-northeast-2(서울) 리전에서 리소스를 생성해야 합니다.

3-1. VPC, EC2

선수는 제공된 docdb_client.py DocumentDB Client Application Python 코드를 수정하지 않고 배포합니다. retail_dataset.json은 DocumentDB 적재용 데이터입니다.

- VPC/Subnet 구성: Client EC2와 DocumentDB Cluster가 통신 가능하고, Client EC2는 Public IP와 TCP/8080으로 외부 접근 가능해야 함
- DocumentDB Cluster: 외부 직접 노출 금지
- Client EC2 Name Tag: skills-nosql-client-ec2
- Client Application: 제공 Python 소스코드를 수정하지 않고 배포
- Client Application Listen Address / Port: 0.0.0.0 / TCP 8080
- Client Application 고정값: AWS Region ap-northeast-2, Secret Name skills-nosql-docdb-secret, Database skills_retail, DocumentDB Port 27017, TLS Enabled
- Client Application 실행 시 boto3, pymongo 패키지와 Amazon DocumentDB TLS CA Bundle(global-bundle.pem)을 사용할 수 있어야 함

3-2. DocumentDB

- DocumentDB Cluster Identifier: skills-nosql-docdb-cluster
- DocumentDB Instance Identifier: skills-nosql-docdb-instance-1
- DocumentDB Instance Class: db.t3.medium
- Storage Encryption / KMS Key Alias: Enabled / alias/skills-nosql-docdb
- TLS: Enabled
- Backup Retention Period: 1일 이상

3-3. Data Model, Collections, Data Field, Index and TTL

선수는 제공된 retail_dataset.json 데이터를 DocumentDB에 적재합니다. 인덱스 이름은 자유롭게 지정할 수 있으나, 컬렉션, 필드, 정렬 방향, Unique 여부, TTL 옵션은 아래 조건을 만족해야 합니다.

- Database Name: skills_retail
- Collections / 최소 데이터 수: orders 8개 이상, products 6개 이상, sessions 3개 이상
- BSON Date 필드: orders.createdAt, orders.dueAt, products.updatedAt, sessions.lastSeen, sessions.expiresAt
- orders: { orderId: 1 } unique, { customerId: 1, createdAt: -1 }, { status: 1, dueAt: 1 }
- products: { productId: 1 } unique, { warehouseId: 1, stock: 1 }
- sessions: { sessionId: 1 } unique, { expiresAt: 1 } TTL expireAfterSeconds: 0, {

customerId: 1, lastSeen: -1 }

3-4. Secrets Manager

- Secret Name: skills-nosql-docdb-secret
- Required Keys: username, password, host(DocumentDB Cluster Endpoint Hostname이며 Scheme 또는 Port를 포함하지 않음)

4. Simplify Service Networking with VPC Lattice

두 Python 애플리케이션(Client, Service)을 서로 다른 VPC의 EC2에 배포하고, VPC Lattice Target Group, Service, Listener, Service Network로 연결합니다. 본 모듈은 ap-northeast-1(도쿄) 리전에 리소스를 생성하며, Client VPC와 Service VPC 사이에 VPC Peering 또는 Transit Gateway 직접 네트워킹 연결을 구성하지 않습니다.



4-1. VPC

- Client VPC Name Tag / CIDR: skills-lattice-client-vpc / 10.61.0.0/16
- Service VPC Name Tag / CIDR: skills-lattice-service-vpc / 10.62.0.0/16
- Client EC2는 Public IP로 HTTP 접근 가능, Service EC2는 Public IP 없이 내부 서비스로 구성

4-2. Client EC2 Application

- EC2 Name Tag: skills-lattice-client-ec2
- Application File: 제공된 client_app.py
- Listen Port: TCP/80
- Environment: SERVICE_URL은 skills-lattice-order-service의 VPC Lattice Generated Domain 기반 URL로 설정
- Required API: GET /health는 HTTP 200, GET /v1/client/orders?id=1001은 VPC Lattice를 통해 Service 앱을 호출하고 service.order_id=1001, service.via=vpc-lattice를 포함해야 함
- Security Group: TCP/80 from 0.0.0.0/0 Inbound 허용, VPC Lattice Service Domain으로 Outbound 요청 가능

4-3. Service EC2 Application

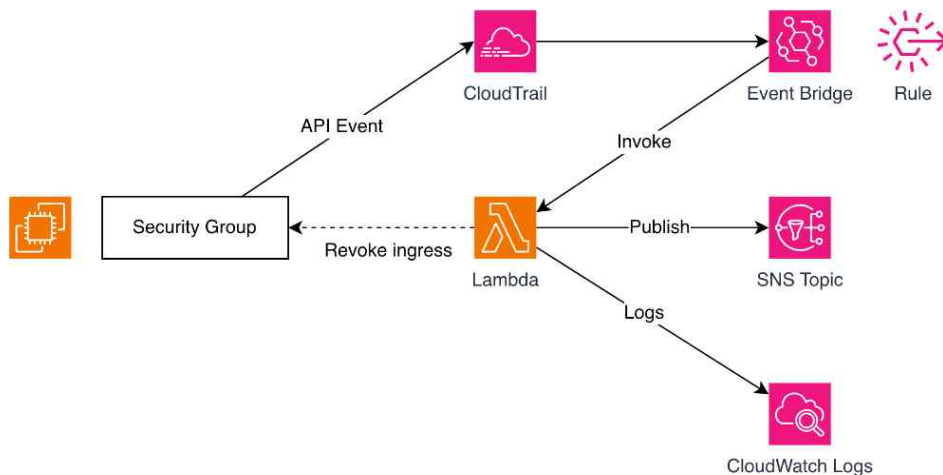
- EC2 Name Tag: skills-lattice-service-ec2
- Application File: 제공된 service_app.py
- Listen Port: TCP/8080
- Required API: GET /health는 HTTP 200, GET /v1/orders?id=1001은 order_id=1001, via=vpc-lattice를 포함해야 함
- Security Group: TCP/8080은 VPC Lattice Managed Prefix List 소스만 허용하며, 0.0.0.0/0 허용 시 미충족

4-4. VPC Lattice Service Network

- Service Network Name Tag: skills-lattice-sn
- VPC Association: Client VPC skills-lattice-client-vpc 연결, Association Security Group은 TCP/80 from Client VPC CIDR 10.61.0.0/16 허용
- Service Name Tag: skills-lattice-order-service, Service Association은 skills-lattice-sn에 연결
- Target Group Name Tag: skills-lattice-order-tg, Type Instance, HTTP/8080, VPC skills-lattice-service-vpc, Health Check HTTP /health, Target skills-lattice-service-ec2
- Listener Name Tag: skills-lattice-http-listener, HTTP/80, Default Action은 skills-lattice-order-tg로 Forward

5. Cloud Event Handling

보호 대상 Security Group에 Inbound 규칙이 추가되면 이를 감지하여 알림을 보내고 원래 상태로 복구하는 자동화를 구성합니다. 본 모듈은 ap-southeast-1(싱가포르) 리전에 리소스를 생성해야 합니다.



5-1. VPC, EC2

- VPC Name Tag / CIDR: skills-ceh-vpc / 10.73.0.0/16
- EC2 Name Tag: skills-ceh-ec2
- EC2 Security Group Name Tag: skills-ceh-protected-sg
- skills-ceh-protected-sg는 최종 제출 시점에 Inbound 규칙이 0개여야 함

5-2. SNS Topic

- Topic Name Tag: skills-ceh-alert-topic
- Topic Type: Standard

5-3. Lambda

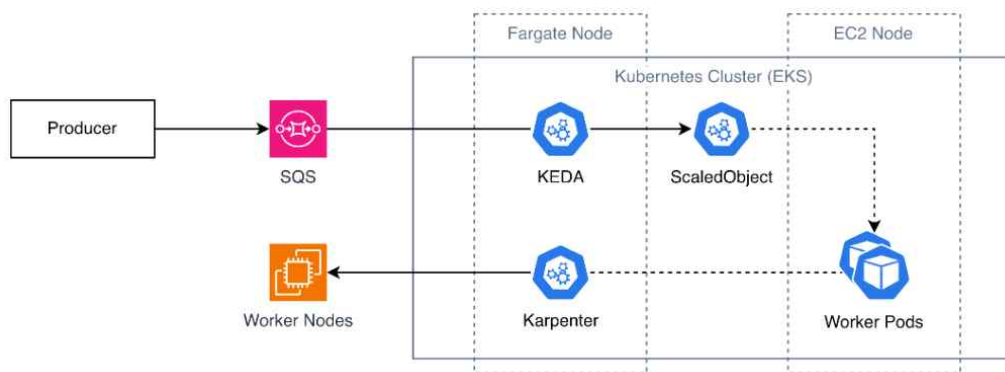
- Function Name Tag: skills-ceh-remediate-fn
- Application File: 제공된 remediate_security_group.py
- Runtime / Handler / Timeout: Python 3.12 / remediate_security_group.lambda_handler / 30초 이상
- Environment: PROTECTED_SECURITY_GROUP_ID=skills-ceh-protected-sg의 Security Group ID, SNS_TOPIC_ARN=skills-ceh-alert-topic의 ARN
- IAM 권한: Security Group 조회/수정, SNS 발행, CloudWatch Logs 기록 가능

5-4. EventBridge Rule

- Rule Name Tag: skills-ceh-sg-change-rule
- CloudTrail Trail Name: skills-ceh-cloudtrail
- CloudTrail Logging: Enabled
- Event Bus: default
- Event Target : Lambda, skills-ceh-remediate-fn
- Event Pattern: AWS API Call via CloudTrail 중 EC2 AuthorizeSecurityGroupIngress 이벤트 감지
- 동작: Inbound 규칙 0개로 복구, skills-ceh-alert-topic 알림 발행, /aws/lambda/skills-ceh-remediate-fn 로그 기록

6. Event-driven Pod Scaling with AWS SQS

SQS Queue 길이에 따라 Worker Pod가 증가/감소하고, Karpenter가 EC2 Worker Node를 동적으로 스케일링하는 구조를 구현합니다. 본 모듈은 us-west-2(오레곤) 리전에 리소스를 생성해야 합니다.



6-1. VPC, EKS Cluster

- VPC/Subnet 구성: EKS Cluster, Fargate Profile, Karpenter EC2 Worker Node가 생성되고 Worker Pod에서 SQS API 호출이 성공해야 함
- EKS Cluster Name: skills-sqs-cluster
- EKS Cluster Endpoint: CloudShell에서 kubectl 명령으로 접근 가능해야 함

6-2. Fargate Profile

- skills-sqs-fp-keda: Selector Namespace keda
- skills-sqs-fp-karpenter: Selector Namespace karpenter

6-3. SQS Queue

- SQS Queue Name: skills-sqs-queue
- Queue Type / Visibility Timeout: Standard / 30초 이상

6-4. IAM IRSA

아래 ServiceAccount에는 eks.amazonaws.com/role-arn annotation이 있어야 하며, 해당 Role은 각 컨트롤러와 Worker 기능 수행에 사용됩니다.

- keda/keda-operator, karpenter/karpenter, skills-sqs/sqs-worker-sa

6-5. Worker Application

- Application File: 제공된 worker.py
- Worker 컨테이너 이미지는 제공된 worker.py와 boto3 의존성을 포함해야 하며, Dockerfile은 선수 구현 방식에 맞게 직접 작성합니다.
- Namespace / Deployment / ServiceAccount: skills-sqs / sqs-worker / sqs-worker-sa
- Label / Selector: app=sqs-worker
- Required Environment: SQS_QUEUE_URL, AWS_REGION, PROCESSING_SECONDS(5 권장)
- Node Selector: karpenter.sh/nodepool=skills-sqs-nodepool, skills-nodepool=event-worker
- Worker Pod는 Fargate Node가 아닌 Karpenter EC2 Worker Node에서 실행되어야 함

6-6. KEDA

- Namespace / Operator Deployment: keda / keda-operator
- ScaledObject / TriggerAuthentication: sqs-worker-scaledobject / sqs-worker-trigger-auth (skills-sqs Namespace)
- Scale Target: Deployment sqs-worker
- Trigger: aws-sqs-queue, queueLength 2, min 0, max 6, pollingInterval 15초 이하, cooldownPeriod 30초 이하

6-7. Karpenter

- Namespace / Controller Deployment: karpenter / karpenter
- NodePool / EC2NodeClass: skills-sqs-nodepool / skills-sqs-nodeclass
- NodePool Label: skills-nodepool=event-worker
- NodePool에는 spec.disruption.consolidationPolicy 설정 포함